

Lab 04: Spark Streaming, Kafka và MongoDB

May 2025

Table of Contents

1	Extract	4
1.1	Các thư viện (Python) cần thiết khi kết nối Kafka	4
1.2	Workflow tổng thể	4
2	Transform	5
2.1	Transform – Moving Statistics	5
2.1.1	Workflow tổng thể	5
2.2	Transform – Moving Z-score	7
2.2.1	Các thư viện (JAR) cần thiết khi kết nối Kafka	7
2.2.2	Workflow tổng thể	7
3	Load	10
3.1	Các thư viện (JAR) cần thiết khi kết nối Kafka	10
3.2	Các thư viện (JAR) cần thiết khi kết nối MongoDB	10
3.3	Cấu hình Dockerfile	10
3.4	Workflow tổng thể	11
3.5	Tích hợp vào Docker Compose	15
3.6	How to Run Load	15
4	Bonus	16
4.1	Các thư viện (JAR) cần thiết khi kết nối Kafka	16
4.2	Workflow tổng thể	16

Team's Contribution

StudentID	Member	Task	Contribution
22120210	Lê Võ Nhật Minh	Transform Report	Thực hiện tính toán Z-score và ghi vào topic btc-price-zscore Chi tiết cách triển khai thuật toán và chuẩn bị môi trường
22120227	Nguyễn Hữu Nghĩa	Transform Report	Thực hiện tính toán các chỉ số thống kê của giá Bitcoin và ghi vào topic btc-price-moving Chi tiết cách triển khai thuật toán và chuẩn bị môi trường
22120262	Nguyễn Lê Tấn Phát	Load Report	Viết mã nguồn đọc từ topic btc-price-zscore và thực hiện ghi vào collection theo từng window như đề bài yêu cầu Chi tiết cách triển khai thuật toán và chuẩn bị môi trường
22120405	Nguyễn Thanh Tuấn	Extract Bonus Report	Thực hiện gọi API lấy dữ liệu và ghi vào topic btc-price Với mỗi bản ghi, tính window đầu tiên lớn/nhỏ hơn giá trị bản ghi đó Chi tiết cách triển khai thuật toán và chuẩn bị môi trường

1 Extract

1.1 Các thư viện (Python) cần thiết khi kết nối Kafka

- `kafka-python`
→ Thư viện client chính để gửi dữ liệu từ Python tới Kafka (sử dụng `KafkaProducer`, `KafkaAdminClient`).
- `python-dotenv`
→ Cho phép đọc biến môi trường từ file `.env`, thuận tiện cấu hình.
- `Flask`
→ Dùng để triển khai web server đơn giản nhằm kiểm tra hoặc expose trạng thái hệ thống.
- `requests`
→ Thư viện phổ biến để gọi HTTP API và nhận dữ liệu từ endpoint bên ngoài.

1.2 Workflow tổng thể

Bước 1: Kiểm tra kết nối Kafka trước khi chạy

- Trước khi gửi dữ liệu, hệ thống sử dụng `KafkaAdminClient` để thử kết nối tới broker (host và port lấy từ `KAFKA_HOST`, `KAFKA_PORT`).
- Nếu kết nối thất bại liên tục vượt quá `KAFKA_MAX_RETRIES`, hệ thống sẽ dừng.

Bước 2: Khởi động Flask server và luồng crawl dữ liệu

- Flask server lắng nghe tại địa chỉ `0.0.0.0` và cổng do biến môi trường `APP_PORT` chỉ định (mặc định `5000`).
- Hàm `get_btcdst_value` chạy ở thread song song, thực hiện crawl dữ liệu liên tục từ API được cấu hình với khoảng cách thời gian mỗi lần là `DATA_CRAWL_INTERVAL` (mặc định: `50ms`).

Bước 3: Crawl dữ liệu giá BTC/USDT

- Endpoint API được lấy từ biến môi trường `DATA_CRAWL_API_URL`.
- Nếu gọi thành công (status 200), dữ liệu được gắn thêm timestamp định dạng chuẩn ISO 8601 UTC.
- Timestamp được làm tròn theo bội số của `DATA_CRAWL_INTERVAL` để đồng bộ với xử lý Spark phía sau.
- Nếu lỗi (timeout, không truy cập được,...), sẽ retry tối đa `DATA_CRAWL_MAX_RETRIES`.

Bước 4: Gửi dữ liệu vào Kafka

- Mỗi bản tin (JSON) được gửi tới topic do biến `KAFKA_TOPIC` quy định (`btc-price`).
- Mỗi record bao gồm:
 - `symbol`: tên loại tiền tệ (`"BTCUSDT"`).
 - `price`: giá trị hiện tại.
 - `timestamp`: thời điểm crawl dữ liệu.
- Kafka key: theo symbol để đảm bảo partition trong topic giữ đúng thứ tự.
- Sau khi đủ số lần retry (hoặc kết thúc crawl), Kafka producer sẽ được `flush()` và đóng kết nối.

Hình dữ liệu trong topic `btc-price`:

Offset	Partition	Timestamp	Key	Value
0	2	6/17/2025, 23:33:48	BTCUSD	{\"symbol\": \"BTCUSD\", \"price\": \"104151.97000000...\"}
1	2	6/17/2025, 23:33:48	BTCUSD	{\"symbol\": \"BTCUSD\", \"price\": \"104151.97000000...\"}
2	2	6/17/2025, 23:33:48	BTCUSD	{\"symbol\": \"BTCUSD\", \"price\": \"104151.97000000...\"}
3	2	6/17/2025, 23:33:48	BTCUSD	{\"symbol\": \"BTCUSD\", \"price\": \"104151.97000000...\"}
4	2	6/17/2025, 23:33:48	BTCUSD	{\"symbol\": \"BTCUSD\", \"price\": \"104151.96000000...\"}
5	2	6/17/2025, 23:33:49	BTCUSD	{\"symbol\": \"BTCUSD\", \"price\": \"104151.97000000...\"}
6	2	6/17/2025, 23:33:49	BTCUSD	{\"symbol\": \"BTCUSD\", \"price\": \"104151.97000000...\"}
7	2	6/17/2025, 23:33:49	BTCUSD	{\"symbol\": \"BTCUSD\", \"price\": \"104151.97000000...\"}
8	2	6/17/2025, 23:33:49	BTCUSD	{\"symbol\": \"BTCUSD\", \"price\": \"104151.97000000...\"}
9	2	6/17/2025, 23:33:49	BTCUSD	{\"symbol\": \"BTCUSD\", \"price\": \"104151.97000000...\"}

Figure 1: Các records trong topic `btc-price`.

2 Transform

2.1 Transform – Moving Statistics

Các chương trình Spark từ đây về sau sử dụng thư viện tương tự:

- `spark-sql-kafka-0-10_2.12-<version>.jar`
→ Kết nối chính giữa Spark Structured Streaming và Kafka.
- `kafka-clients-<version>.jar`
→ Client Kafka giúp Spark đọc/ghi dữ liệu từ broker.
- `spark-token-provider-kafka-0-10_2.12-<version>.jar`
→ Hỗ trợ bảo mật, bắt buộc khi dùng Spark-Kafka connector.
- `commons-pool2-<version>.jar`
→ Thư viện phụ trợ, tránh lỗi thiếu class trong một số version Spark.

2.1.1 Workflow tổng thể

Bước 1: Đọc dữ liệu từ Kafka

- Topic nguồn: `btc-price` do `extract-service` đẩy dữ liệu vào.
- Spark Structured Streaming:
 - Sử dụng `.format("kafka"), .option("subscribe", "btc-price")`.
 - Kafka broker: `broker:9092` (nếu chạy cùng Docker) hoặc `localhost:9094` (nếu Spark chạy bên ngoài container).

Bước 2: Parse và chuẩn hóa dữ liệu

- Parse JSON value thành struct với các trường: `symbol`, `price`, `timestamp`.
- Ép kiểu dữ liệu:
 - `price`: chuyển về `double`.

- `timestamp`: ép về `TimestampType` với định dạng `yyyy-MM-dd'T'HH:mm:ss.SSS'Z'`. (định dạng ISO8061 UTC timestamp)
- Lọc dữ liệu lỗi: loại bỏ các record có `price` là `null` hoặc `timestamp` sai định dạng.

Bước 3: Xử lý Event-time & Watermark

- Event-time: sử dụng trường `ts` đã chuẩn hóa để làm thời gian sự kiện.
- Watermark: thiết lập "10 seconds" để chấp nhận trễ tối đa 10 giây, đúng yêu cầu đề bài.

Bước 4: Tính toán thống kê theo nhiều cửa sổ thời gian

- Các cửa sổ cần xử lý: 30s, 1m, 5m, 15m, 30m, 1h.
- Thống kê: tính `avg(price)` và `stddev(price)` trong mỗi cửa sổ.
- Gán thêm cột "window" để phân biệt mỗi cửa sổ, ví dụ: "30s", "1m", ...
- Xử lý trường hợp `stddev = null`:
 - Với các cửa sổ chỉ có 1 bản ghi, Spark không đủ dữ liệu để tính độ lệch chuẩn → `std_price = null`.
 - Để đảm bảo output hợp lệ, xử lý bằng cách thay `null` bằng 0.0 khi ghi vào Kafka.

Bước 5: Tổng hợp và chuẩn hóa output

- Gộp toàn bộ kết quả từ các cửa sổ vào một `DataFrame` chung.
- Group theo (`symbol`, `emit_ts`) để gom tất cả các cửa sổ tại cùng thời điểm thành mảng.
- Chuyển đổi về chuỗi JSON đúng format yêu cầu:

```
{
  "timestamp": "2025-06-14T10:33:00.000Z",
  "symbol": "BTCUSDT",
  "windows": [
    {
      "window": "30s",
      "avg_price": 105069.46,
      "std_price": 0.0050
    },
    ...
  ]
}
```

- `emit_ts`: được lấy từ `window.start`, vì trong phần extract timestamp đã được làm tròn theo khoảng thời gian → đảm bảo khớp timestamp gốc để tính Z-score chính xác.

Hình minh họa output:

```

{
  "timestamp": "2025-06-16T10:52:00.000Z",
  "symbol": "BTCUSD1",
  "windows": [
    {
      "window": "30s",
      "avg_price": 106793.74788821917,
      "std_price": 14.323272461888036
    },
    {
      "window": "1m",
      "avg_price": 106793.74788821917,
      "std_price": 14.323272461888036
    },
    {
      "window": "30s",
      "avg_price": 106779.52999999993,
      "std_price": 18.009205027230475
    },
    {
      "window": "1m",
      "avg_price": 106779.52999999993,
      "std_price": 18.009205027230475
    },
    {
      "window": "30s",
      "avg_price": 106777.72094339617,
      "std_price": 17.69690943451411
    },
    {
      "window": "1m",
      "avg_price": 106774.70480198012,
      "std_price": 16.732820229865954
    }
  ]
}

```

Figure 2: Output thực tế từ topic `btc-price-moving`.

Bước 6: Ghi kết quả ra Kafka

- Topic đích: `btc-price-moving`
- Output mode: `update` – cho phép Spark ghi kết quả cập nhật liên tục cho mỗi cửa sổ.
- Cấu hình thêm:
 - `kafka.bootstrap.servers`: địa chỉ kết nối Kafka.
 - `checkpointLocation`: khai báo đường dẫn để Spark checkpoint, phục hồi trạng thái sau khi khởi động lại.

2.2 Transform – Moving Z-score

Code của phần này được thực hiện trong notebook, do đó toàn bộ quá trình thực thi mã nguồn được thực hiện bằng cách ấn run từng cell thủ công

2.2.1 Các thư viện (JAR) cần thiết khi kết nối Kafka

- `spark-sql-kafka-0-10_2.12-<version>.jar`
→ Kết nối chính giữa Spark Structured Streaming và Kafka.
- `kafka-clients-<version>.jar`
→ Client Kafka giúp Spark đọc/ghi dữ liệu từ broker.
- `spark-token-provider-kafka-0-10_2.12-<version>.jar`
→ Hỗ trợ bảo mật, bắt buộc khi dùng Spark-Kafka connector.
- `commons-pool2-<version>.jar`
→ Thư viện phụ trợ, tránh lỗi thiếu class trong một số version Spark.

2.2.2 Workflow tổng thể

Bước 1: Đọc dữ liệu từ Kafka

- Topic nguồn: `btc-price` do `extract-service` và `btc-price-moving` do `transform-moving` đẩy dữ liệu vào.

- Spark Structured Streaming:
 - Sử dụng `.format("kafka"), .option("subscribe", "btc-price")`.
 - Kafka broker: `broker:9092` (nếu chạy cùng Docker) hoặc `localhost:9094` (nếu Spark chạy bên ngoài container).

Bước 2: Parse và chuẩn hóa dữ liệu

- Đối với topic `btc-price`:
 - Parse trường `value` (dạng JSON) thành struct gồm các trường: `symbol`, `price`, `timestamp`.
 - Ép kiểu dữ liệu:
 - * `price`: chuyển về kiểu `double`.
 - * `timestamp`: ép về kiểu `TimestampType` với định dạng `yyyyMMdd'T'HHmmss.SSS'Z'`.
 - Loại dữ liệu lỗi: loại bỏ các bản ghi có `price` là `null` hoặc `timestamp` sai định dạng.
- Đối với topic `btc-price-moving`:
 - Parse trường `value` (dạng JSON) thành struct với các trường: `symbol`, `timestamp`, `windows`.
 - Lưu ý: trường `windows` là một mảng các struct, mỗi phần tử gồm: `window`, `avg_price`, `std_price`.
 - Ép kiểu dữ liệu:
 - * `symbol`: chuyển về kiểu `string`.
 - * `timestamp`: parse từ định dạng ISO 8601 UTC và ép về `TimestampType`.
 - * `windows`: ép về kiểu `array` rồi thực hiện `explode` để tách từng phần tử trong mảng.
 - Loại dữ liệu lỗi: loại bỏ các bản ghi có trường `avg_price` hoặc `timestamp` bị lỗi hoặc thiếu.

Bước 3: Xử lý Event-time & Watermark đối với topic `btc-price-moving`

- Event-time: sử dụng trường `ts` đã chuẩn hóa để làm thời gian sự kiện.
- Watermark: thiết lập `"10 seconds"` để chấp nhận trễ tối đa 10 giây, đúng yêu cầu đề bài.

Bước 4: Tính toán thống kê theo nhiều cửa sổ thời gian

- Khai báo danh sách các cửa sổ thời gian trượt (*sliding windows*) với nhiều khoảng khác nhau: `"30s"`, `"1m"`, `"5m"`, `"15m"`, `"30m"`, `"1h"`.
- Duyệt qua từng loại cửa sổ thời gian trong danh sách.
- Áp dụng cửa sổ thời gian tương ứng vào dữ liệu giá để tạo cột `window` gồm `start` và `end` của mỗi cửa sổ.
- Loại bảng chứa moving average (`flat_moving_df`) theo loại cửa sổ đang xét.
- Thực hiện phép `join` giữa bảng giá và bảng moving average theo 2 điều kiện:
 - Cùng mã cổ phiếu (`symbol = moving_sym`).
 - Thời gian bắt đầu của cửa sổ giá bằng `timestamp` của moving average.
- Tính Z-score với công thức:

$$Z = \begin{cases} 0 & \text{if std} = 0 \\ \frac{\text{price} - \text{mean}}{\text{std}} & \text{otherwise} \end{cases}$$

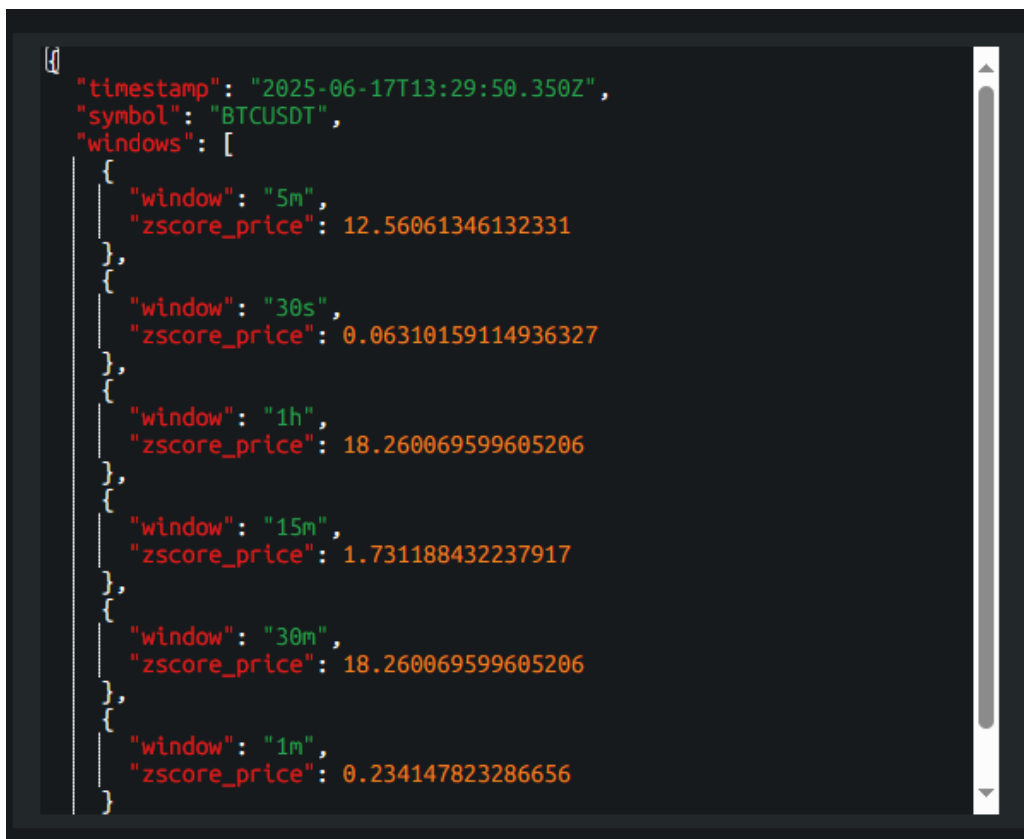
- Chọn các cột cần thiết: `timestamp`, `symbol`, `window_type`, `zscore_price`.
- Gộp kết quả từ tất cả các cửa sổ lại thành một DataFrame duy nhất.
- Loại bỏ các bản ghi trùng lặp theo nhóm (`timestamp`, `symbol`, `window_type`) bằng cách lấy giá trị Z-score lớn nhất trong mỗi nhóm.

Bước 5: Tổng hợp và chuẩn hóa output

- Gộp toàn bộ kết quả từ các cửa sổ vào một DataFrame chung.
- Group theo (`symbol`, `timestamp`) để gom tất cả các cửa sổ tại cùng thời điểm thành mảng.
- Chuyển đổi về chuỗi JSON đúng format yêu cầu:

```
{
  "timestamp": "2025-06-17T13:29:50.350Z",
  "symbol": "BTCUSDT",
  "windows": [
    {
      "window": "5m",
      "zscore_price": 12.56061346132331
    },
    ...
  ]
}
```

Hình minh họa output:



```
"timestamp": "2025-06-17T13:29:50.350Z",
"symbol": "BTCUSDT",
"windows": [
  {
    "window": "5m",
    "zscore_price": 12.56061346132331
  },
  {
    "window": "30s",
    "zscore_price": 0.06310159114936327
  },
  {
    "window": "1h",
    "zscore_price": 18.260069599605206
  },
  {
    "window": "15m",
    "zscore_price": 1.731188432237917
  },
  {
    "window": "30m",
    "zscore_price": 18.260069599605206
  },
  {
    "window": "1m",
    "zscore_price": 0.234147823286656
  }
]
```

Figure 3: Output thực tế từ topic `btc-price-zscore`.

Bước 6: Ghi kết quả ra Kafka

- Topic đích: `btc-price-zscore`
- Output mode: `append` – Spark chỉ ghi những bản ghi mới được thêm vào kể từ batch trước
- Cấu hình thêm:
 - `kafka.bootstrap.servers`: địa chỉ kết nối Kafka.
 - `checkpointLocation`: khai báo đường dẫn để Spark checkpoint, phục hồi trạng thái sau khi khởi động lại.

3 Load

3.1 Các thư viện (JAR) cần thiết khi kết nối Kafka

- `spark-sql-kafka-0-10_2.12-<version>.jar`
→ Kết nối chính giữa Spark Structured Streaming và Kafka.
- `kafka-clients-<version>.jar`
→ Client Kafka giúp Spark đọc/ghi dữ liệu từ broker.
- `spark-token-provider-kafka-0-10_2.12-<version>.jar`
→ Hỗ trợ bảo mật, bắt buộc khi dùng Spark-Kafka connector.
- `commons-pool2-<version>.jar`
→ Thư viện phụ trợ, tránh lỗi thiếu class trong một số version Spark.

3.2 Các thư viện (JAR) cần thiết khi kết nối MongoDB

- `mongo-spark-connector_2.13-10.5.0.jar`
→ Kết nối Spark với MongoDB thông qua connector chính thức.
- `mongodb-driver-sync-4.10.2.jar`
→ Driver MongoDB hỗ trợ thao tác dữ liệu đồng bộ trong Python.
- `bson-4.10.2.jar`
→ Hỗ trợ serialize/deserialize BSON (Binary JSON), format mặc định của MongoDB.
- `mongodb-driver-core-4.10.2.jar`
→ Thư viện lõi của driver MongoDB.

3.3 Cấu hình Dockerfile

Đường dẫn thư mục làm việc: `/app/load`

Thêm các thư viện: tải các file `.jar` từ Maven và thêm vào thư mục `/opt/spark/jars/`.

Chạy file: `22120210.py` bằng `spark-submit` cùng với tất cả các JAR phụ thuộc cần thiết.

```
CMD ["spark-submit", "--jars", "/opt/spark/jars/spark-sql-kafka.jar,...",  
     "/app/load/22120210.py"]
```

3.4 Workflow tổng thể

Bước 1: Khởi tạo SparkSession

- Tên ứng dụng: BTC Z-Score Loader
- Kết nối MongoDB qua URI: `mongodb://mongodb:27017/`

Bước 2: Đọc dữ liệu từ Kafka

- Topic: `btc-price-zscore`
- Định dạng: `.format("kafka")`
- Một số tùy chọn:
 - `startingOffsets = latest`
 - `maxOffsetsPerTrigger = 1000` (Để phòng lỗi tràn bộ nhớ heap)

Bước 3: Parse dữ liệu JSON

- Schema gồm 3 trường: `timestamp`, `symbol`, `windows` (array)
- Bóc tách từng phần tử trong mảng `windows` bằng hàm `explode()`
- Trường cần lấy: `timestamp`, `symbol`, `window`, `zscore_price`

Bước 4: Ghi dữ liệu vào MongoDB

- Với mỗi bản ghi, xác định `window` để ghi vào collection tương ứng.
- Tên collection: `btc-price-zscore-{window}`
- Cấu trúc document MongoDB:

```
{
  "timestamp": ISODate(...),
  "symbol": "BTCUSDT",
  "zscore_price": 1.23
}
```

- Dữ liệu được ghi bằng `pymongo`.
- Dùng `foreachBatch` để ghi mỗi batch dữ liệu một cách linh hoạt và có kiểm soát, tránh tràn bộ nhớ.

```
PS D:\work\introduction-to-big-data\lab 4\src> docker exec -it mongod mongosh
Current Mongosh Log ID: 68517d2a738f6226d9e327
Connecting to: mongod://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.2
Using Mongod: 8.0.10
Using Mongosh: 2.5.2
For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2025-06-17T13:36:48.172400: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2025-06-17T13:36:49.439400: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2025-06-17T13:36:49.439400: For customers running the current memory allocator, we suggest changing the contents of the following sysfsfile
2025-06-17T13:36:49.439400: For customers running the current memory allocator, we suggest changing the contents of the following sysfsfile
2025-06-17T13:36:49.439400: We suggest setting the contents of sysfsfile to 0.
2025-06-17T13:36:49.439400: vm.max_map_count is too low
2025-06-17T13:36:49.439400: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
-----

test> use btc
switched to db btc
btc> show collections
btc-price-zscore-15m
btc-price-zscore-1h
btc-price-zscore-1m
btc-price-zscore-30m
btc-price-zscore-30s
btc-price-zscore-5m
btc>
```

Figure 4: Các collections trong MongoDB.

```
test> use btc
switched to db btc
btc> show collections
btc-price-zscore-15m
btc-price-zscore-1h
btc-price-zscore-1m
btc-price-zscore-30m
btc-price-zscore-30s
btc-price-zscore-5m
btc> db["btc-price-zscore-15m"].find().limit(5).pretty()
[
  {
    _id: ObjectId('68517d569079399d333df6a4'),
    timestamp: '2025-06-17T14:03:44.000Z',
    symbol: 'BTCUSD',
    zscore_price: -0.23274078046384758
  },
  {
    _id: ObjectId('68517d569079399d333df6e0'),
    timestamp: '2025-06-17T14:06:25.000Z',
    symbol: 'BTCUSD',
    zscore_price: -1.880692342684328
  },
  {
    _id: ObjectId('68517d569079399d333df6e6'),
    timestamp: '2025-06-17T14:03:38.000Z',
    symbol: 'BTCUSD',
    zscore_price: -0.47872261928022093
  },
  {
    _id: ObjectId('68517d569079399d333df6f6'),
    timestamp: '2025-06-17T14:04:11.000Z',
    symbol: 'BTCUSD',
    zscore_price: -0.023193479417839967
  },
  {
    _id: ObjectId('68517d569079399d333df6fd'),
    timestamp: '2025-06-17T14:09:00.000Z',
    symbol: 'BTCUSD',
    zscore_price: 3.1929456879139706
  }
]
btc>
```

Figure 5: Các records trong collection **btc-price-zscore-15m**.

```
zscore_price: -0.023193479417839967
},
{
  _id: ObjectId('68517d569079399d333df61d'),
  timestamp: '2025-06-17T14:09:00.000Z',
  symbol: 'BTCUSD',
  zscore_price: 3.1929456879139706
}
]
btc> db["btc-price-zscore-1h"].find().limit(5).pretty()
[
  {
    _id: ObjectId('68517d569079399d333df602'),
    timestamp: '2025-06-17T14:03:44.000Z',
    symbol: 'BTCUSD',
    zscore_price: -0.23274878846384758
  },
  {
    _id: ObjectId('68517d569079399d333df60b'),
    timestamp: '2025-06-17T14:06:25.000Z',
    symbol: 'BTCUSD',
    zscore_price: -1.380692342684328
  },
  {
    _id: ObjectId('68517d569079399d333df613'),
    timestamp: '2025-06-17T14:03:38.000Z',
    symbol: 'BTCUSD',
    zscore_price: -0.4787226190922093
  },
  {
    _id: ObjectId('68517d569079399d333df618'),
    timestamp: '2025-06-17T14:04:11.000Z',
    symbol: 'BTCUSD',
    zscore_price: -0.023193479417839967
  },
  {
    _id: ObjectId('68517d569079399d333df61a'),
    timestamp: '2025-06-17T14:09:00.000Z',
    symbol: 'BTCUSD',
    zscore_price: 3.1929456879139706
  }
]
btc>
```

Figure 6: Các records trong collection **btc-price-zscore-1h**.

```
zscore_price: -0.023193479417839967
},
{
  _id: ObjectId('68517d569079399d333df61a'),
  timestamp: '2025-06-17T14:09:00.000Z',
  symbol: 'BTCUSD',
  zscore_price: 3.1929456879139706
}
]
btc> db["btc-price-zscore-1m"].find().limit(5).pretty()
[
  {
    _id: ObjectId('68517d569079399d333df605'),
    timestamp: '2025-06-17T14:03:44.000Z',
    symbol: 'BTCUSD',
    zscore_price: -0.7525374155304226
  },
  {
    _id: ObjectId('68517d569079399d333df60a'),
    timestamp: '2025-06-17T14:06:25.000Z',
    symbol: 'BTCUSD',
    zscore_price: -0.0853758921197659
  },
  {
    _id: ObjectId('68517d569079399d333df610'),
    timestamp: '2025-06-17T14:03:38.000Z',
    symbol: 'BTCUSD',
    zscore_price: -1.4914454268090408
  },
  {
    _id: ObjectId('68517d569079399d333df617'),
    timestamp: '2025-06-17T14:04:11.000Z',
    symbol: 'BTCUSD',
    zscore_price: 0.8070407211811207
  },
  {
    _id: ObjectId('68517d569079399d333df61e'),
    timestamp: '2025-06-17T14:09:00.000Z',
    symbol: 'BTCUSD',
    zscore_price: -1.7207707527762646
  }
]
btc>
```

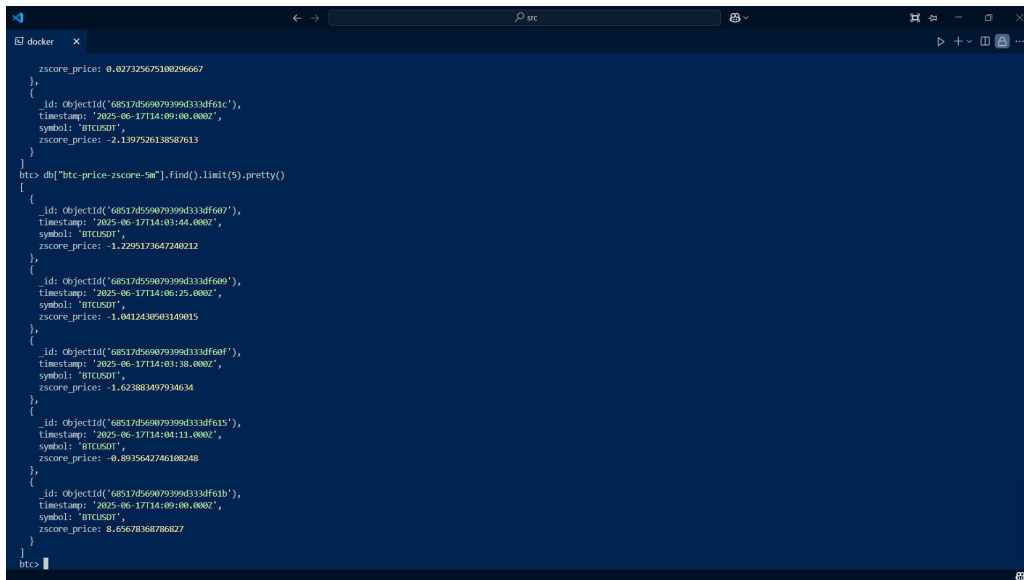
Figure 7: Các records trong collection **btc-price-zscore-1m**.

```
zscore_price: 0.8070407211811207
},
{
  _id: ObjectId('68517d569079399d333df61e'),
  timestamp: '2025-06-17T14:09:00.000Z',
  symbol: 'BTCUSD',
  zscore_price: -1.7207707527762646
}
]
btc> db["btc-price-zscore-30m"].find().limit(5).pretty()
[
  {
    _id: ObjectId('68517d569079399d333df609'),
    timestamp: '2025-06-17T14:03:44.000Z',
    symbol: 'BTCUSD',
    zscore_price: -0.23274078846384758
  },
  {
    _id: ObjectId('68517d569079399d333df608'),
    timestamp: '2025-06-17T14:06:25.000Z',
    symbol: 'BTCUSD',
    zscore_price: -1.380692342684328
  },
  {
    _id: ObjectId('68517d569079399d333df611'),
    timestamp: '2025-06-17T14:03:38.000Z',
    symbol: 'BTCUSD',
    zscore_price: -0.47872261920922093
  },
  {
    _id: ObjectId('68517d569079399d333df614'),
    timestamp: '2025-06-17T14:04:11.000Z',
    symbol: 'BTCUSD',
    zscore_price: -0.023193479417839967
  },
  {
    _id: ObjectId('68517d569079399d333df61f'),
    timestamp: '2025-06-17T14:09:00.000Z',
    symbol: 'BTCUSD',
    zscore_price: 3.1929456879139706
  }
]
btc>
```

Figure 8: Các records trong collection **btc-price-zscore-30m**.

```
zscore_price: -0.023193479417839967
},
{
  _id: ObjectId('68517d569079399d333df61f'),
  timestamp: '2025-06-17T14:09:00.000Z',
  symbol: 'BTCUSD',
  zscore_price: 3.1929456879139706
}
]
btc> db["btc-price-zscore-30s"].find().limit(5).pretty()
[
  {
    _id: ObjectId('68517d569079399d333df606'),
    timestamp: '2025-06-17T14:03:44.000Z',
    symbol: 'BTCUSD',
    zscore_price: -0.11179045463533997
  },
  {
    _id: ObjectId('68517d569079399d333df60d'),
    timestamp: '2025-06-17T14:06:25.000Z',
    symbol: 'BTCUSD',
    zscore_price: 1.2351385172796523
  },
  {
    _id: ObjectId('68517d569079399d333df612'),
    timestamp: '2025-06-17T14:03:38.000Z',
    symbol: 'BTCUSD',
    zscore_price: -1.319797150586232
  },
  {
    _id: ObjectId('68517d569079399d333df619'),
    timestamp: '2025-06-17T14:04:11.000Z',
    symbol: 'BTCUSD',
    zscore_price: 0.027325675100296667
  },
  {
    _id: ObjectId('68517d569079399d333df61c'),
    timestamp: '2025-06-17T14:09:00.000Z',
    symbol: 'BTCUSD',
    zscore_price: -2.1397526138587613
  }
]
btc>
```

Figure 9: Các records trong collection **btc-price-zscore-30s**.



```
zscore_price: 0.027325675100296667
},
{
  _id: ObjectId('68517d569079399d333df61c'),
  timestamp: '2025-06-17T14:09:00.000Z',
  symbol: 'BTCUSD',
  zscore_price: -2.1397526138587613
},
]
btc> db["btc-price-zscore-5m"].find().limit(5).pretty()
[
  {
    _id: ObjectId('68517d569079399d333df607'),
    timestamp: '2025-06-17T14:03:44.000Z',
    symbol: 'BTCUSD',
    zscore_price: -1.2295173647248212
  },
  {
    _id: ObjectId('68517d569079399d333df609'),
    timestamp: '2025-06-17T14:06:25.000Z',
    symbol: 'BTCUSD',
    zscore_price: -1.0412430503149015
  },
  {
    _id: ObjectId('68517d569079399d333df60f'),
    timestamp: '2025-06-17T14:03:38.000Z',
    symbol: 'BTCUSD',
    zscore_price: -1.623883407934634
  },
  {
    _id: ObjectId('68517d569079399d333df615'),
    timestamp: '2025-06-17T14:04:11.000Z',
    symbol: 'BTCUSD',
    zscore_price: -0.8935642746108248
  },
  {
    _id: ObjectId('68517d569079399d333df61b'),
    timestamp: '2025-06-17T14:09:00.000Z',
    symbol: 'BTCUSD',
    zscore_price: 8.65678368786827
  }
]
btc>
```

Figure 10: Các records trong collection **btc-price-zscore-5m**.

3.5 Tích hợp vào Docker Compose

- Service: **load-service**
- Tự động chạy sau khi Kafka và MongoDB sẵn sàng.
- Volume: **spark-checkpoints** mount vào **/tmp** để lưu checkpoint của Spark.

Trích đoạn trong **docker-compose.yml**:

```
load-service:
  build:
    context: ./Load
    dockerfile: Dockerfile
  container_name: load-service
  depends_on:
    kafka:
      condition: service_healthy
    mongodb:
      condition: service_started
  volumes:
    - spark-checkpoints:/tmp
```

3.6 How to Run Load

Sau khi thực thi xong notebook **22120210_zscore.ipynb**, tiến hành thực hiện các bước sau để chạy phase **Load**:

1. **Kiểm tra topic **btc-price-zscore** trong Kafka:**
 - Mở trình duyệt và truy cập địa chỉ: **http://localhost:8082**
 - Vào tab **Topics**, chờ cho đến khi topic **btc-price-zscore** được tạo.
2. **Chạy dịch vụ Load:**

- Mở Terminal và chuyển đến thư mục chứa tệp `docker-compose.yaml`
- Gõ lệnh sau để khởi chạy riêng container `load-service`:

```
docker compose up --build load-service
```

Lưu ý: Đảm bảo rằng các dịch vụ `extract` và `transform` đã được khởi chạy trước và đang gửi dữ liệu vào Kafka. Dịch vụ `load-service` sẽ chỉ bắt đầu ghi dữ liệu vào MongoDB sau khi topic `btc-price-zscore` đã tồn tại.

4 Bonus

4.1 Các thư viện (JAR) cần thiết khi kết nối Kafka

- `spark-sql-kafka-0-10_2.12-<version>.jar`
→ Kết nối chính giữa Spark Structured Streaming và Kafka.
- `kafka-clients-<version>.jar`
→ Client Kafka giúp Spark đọc/ghi dữ liệu từ broker.
- `spark-token-provider-kafka-0-10_2.12-<version>.jar`
→ Hỗ trợ bảo mật, bắt buộc khi dùng Spark-Kafka connector.
- `commons-pool2-<version>.jar`
→ Thư viện phụ trợ, tránh lỗi thiếu class trong một số version Spark.

4.2 Workflow tổng thể

Bước 1: Đọc dữ liệu từ Kafka

- Topic nguồn: `btc-price` do `extract-service` đẩy dữ liệu vào.
- Spark Structured Streaming:
 - Sử dụng `.format("kafka"), .option("subscribe", "btc-price")`.
 - Kafka broker: `broker:9092` (nếu chạy trong Docker).

Bước 2: Parse và chuẩn hóa dữ liệu

- Parse JSON value thành struct với các trường: `symbol`, `price`, `timestamp`.
- Ép kiểu dữ liệu:
 - `price`: ép về `FloatType`.
 - `timestamp`: chuyển về `TimestampType` với định dạng ISO-8601.
- Thiết lập watermark: `30 seconds` (20 giây window và 10 giây trễ) và lưu vào `df`.

Bước 3: Xây dựng cửa sổ trượt động

- Tạo bảng `df_window` chứa `base_price` và khoảng thời gian cửa sổ (`timestamp, timestamp + 20s`] với mỗi bản ghi (`symbol, timestamp`).
- Dùng `dropDuplicates(["symbol", "start_timestamp"])` để tránh trùng lặp.

Bước 4: Tìm mốc thời gian tăng/giảm đầu tiên

- Dùng join giữa `df_window` và `df` theo `symbol` để đưa tất cả các bản ghi vào cửa sổ tương ứng:
 - `value.timestamp > window.start_timestamp`
 - `value.timestamp <= window.end_timestamp`
- Với mỗi hàng dữ liệu sau khi join, tạo thêm cột để kiểm tra thỏa mãn so với `base_price`:
 - `greater_ts`: trả về `price` nếu `price > base_price`, ngược lại trả về `null`.
 - `lesser_ts`: trả về `price` nếu `price < base_price`, ngược lại trả về `null`.
- Dùng `min()` để chọn timestamp đầu tiên lớn hơn/nhỏ hơn trong mỗi nhóm theo `start_timestamp` và lần lượt lưu vào `df_greater` và `df_lesser`.

Bước 5: Tính toán cửa sổ bắt đầu một cửa sổ đến timestamp đầu tiên có giá trị lớn/nhỏ hơn

- Gộp kết quả `df_greater` và `df_lesser` lại với `df_window`.
- Tính toán khoảng thời gian (tính bằng giây) từ `start_timestamp` đến `greater_ts` hoặc `lesser_ts`.
- Nếu không tìm thấy (`null`) thì gán mặc định là `20.0`.
- Format lại timestamp về chuẩn `yyyy-MM-ddTHH:mm:ssZ`.

Bước 6: Ghi kết quả ra Kafka

- Topic đích:
 - `btc-price-higher`: chứa thời gian tăng đầu tiên.
 - `btc-price-lower`: chứa thời gian giảm đầu tiên.

- Cấu trúc JSON mẫu:

```
{
  "timestamp": "2025-06-17T06:20:00Z",
  "higher_window": 7.5
}
{
  "timestamp": "2025-06-17T06:20:00Z",
  "lower_window": 3.1
}
```

- Output mode: `append`
- Cấu hình thêm:
 - `kafka.bootstrap.servers`: `broker:9092`
 - `checkpointLocation`: `/tmp/kafka-checkpoint-higher` và `/tmp/kafka-checkpoint-lower`

Hình minh họa output trong topic `btc-price-higher`:

Offset	Partition	Timestamp	Key	Value
0	1	6/17/2025, 23:34:39	2025-06-17T16:34:00Z	("timestamp":"2025-06-17T16:34:00Z";"higher_wi...
Key Value Headers <pre> {"timestamp": "2025-06-17T16:34:00Z", "higher_window": 16.34999998463257 </pre> Timestamp 6/17/2025, 23:34:39 Timestamp type: CREATE_TIME Key Serde String Size: 20 Bytes Value Serde String Size: 70 Bytes				
0	2	6/17/2025, 23:34:39	2025-06-17T16:33:58Z	("timestamp":"2025-06-17T16:33:58Z";"higher_wi...
1	2	6/17/2025, 23:34:39	2025-06-17T16:33:59Z	("timestamp":"2025-06-17T16:33:59Z";"higher_wi...
2	2	6/17/2025, 23:34:39	2025-06-17T16:33:59Z	("timestamp":"2025-06-17T16:33:59Z";"higher_wi...
1	1	6/17/2025, 23:34:39	2025-06-17T16:34:00Z	("timestamp":"2025-06-17T16:34:00Z";"higher_wi...
3	2	6/17/2025, 23:34:39	2025-06-17T16:33:59Z	("timestamp":"2025-06-17T16:33:59Z";"higher_wi...
2	1	6/17/2025, 23:34:39	2025-06-17T16:34:00Z	("timestamp":"2025-06-17T16:34:00Z";"higher_wi...
4	2	6/17/2025, 23:34:39	2025-06-17T16:33:59Z	("timestamp":"2025-06-17T16:33:59Z";"higher_wi...
5	2	6/17/2025, 23:34:39	2025-06-17T16:33:59Z	("timestamp":"2025-06-17T16:33:59Z";"higher_wi...
3	1	6/17/2025, 23:34:39	2025-06-17T16:34:00Z	("timestamp":"2025-06-17T16:34:00Z";"higher_wi...

Figure 11: Các records trong topic btc-price-higher.

Hình minh họa output trong topic btc_price_lower:

Offset	Partition	Timestamp	Key	Value
0	1	6/17/2025, 23:34:39	2025-06-17T16:34:00Z	("timestamp":"2025-06-17T16:34:00Z";"lower_wi...
Key Value Headers <pre> {"timestamp": "2025-06-17T16:34:00Z", "lower_window": 0.65000000953674316 </pre> Timestamp 6/17/2025, 23:34:39 Timestamp type: CREATE_TIME Key Serde String Size: 20 Bytes Value Serde String Size: 70 Bytes				
0	2	6/17/2025, 23:34:39	2025-06-17T16:33:58Z	("timestamp":"2025-06-17T16:33:58Z";"lower_wi...
1	2	6/17/2025, 23:34:39	2025-06-17T16:33:59Z	("timestamp":"2025-06-17T16:33:59Z";"lower_wi...
2	2	6/17/2025, 23:34:39	2025-06-17T16:33:59Z	("timestamp":"2025-06-17T16:33:59Z";"lower_wi...
1	1	6/17/2025, 23:34:39	2025-06-17T16:34:00Z	("timestamp":"2025-06-17T16:34:00Z";"lower_wi...
3	2	6/17/2025, 23:34:39	2025-06-17T16:33:59Z	("timestamp":"2025-06-17T16:33:59Z";"lower_wi...
2	1	6/17/2025, 23:34:39	2025-06-17T16:34:00Z	("timestamp":"2025-06-17T16:34:00Z";"lower_wi...
4	2	6/17/2025, 23:34:39	2025-06-17T16:33:59Z	("timestamp":"2025-06-17T16:33:59Z";"lower_wi...
5	2	6/17/2025, 23:34:39	2025-06-17T16:33:59Z	("timestamp":"2025-06-17T16:33:59Z";"lower_wi...
3	1	6/17/2025, 23:34:39	2025-06-17T16:34:00Z	("timestamp":"2025-06-17T16:34:00Z";"lower_wi...

Figure 12: Các records trong topic btc-price-lower.